

Jour 1 — Après-midi : Chapitre 2 — Prompting avancé pour développeurs

Niveau : DÉBUTANT — Durée : ~3h30 (après-midi du jour 1)

Pourquoi ce chapitre ?

Vous avez maintenant un assistant IA dans votre IDE. **Mais** : si vous lui dites « *écris une fonction* », vous obtiendrez probablement un résultat **médiocre**. Pourquoi ? Parce que vous ne lui avez **rien expliqué**.

Un prompt mal écrit, c'est comme **un brief flou donné à un freelance** : il fera ce qu'il croit que vous voulez, ce qui n'est presque jamais ce dont vous avez vraiment besoin.

Bonne nouvelle : il existe une **recette** pour écrire des prompts qui marchent. C'est ce qu'on va apprendre.

Objectifs

1. Connaître le canevas **ROCDC-IO** (Rôle, Objectif, Contraintes, DoD, Contexte, I/O).
2. Comprendre quand utiliser un prompt **itérable** ou **idempotent**.
3. Faire **réécrire** un prompt par l'IA elle-même (technique avancée mais simple).
4. Découvrir les **skills** et **subagents** : pour réutiliser vos meilleurs prompts.
5. Écrire des prompts **spéciaux pour générer du code**.

Plan de l'après-midi

| Heure | Section | Durée | |---|---|---| | 14:00 | 1. Glossaire & analogie du freelance | 15 min | | 14:15 | 2. Le canevas ROCDC-IO expliqué pas à pas | 40 min | | 14:55 | 3. Itérable vs idempotent : 2 types de prompts | 25 min | | 15:20 | **Pause** | 15 min | | 15:35 | 4. Méta-prompting : faire écrire ses prompts par l'IA | 25 min | | 16:00 | 5. Skills et subagents : factoriser ses prompts | 20 min | | 16:20 | 6. Prompts spéciaux pour le code | 20 min | | 16:40 | **TP 2** — Construire votre bibliothèque de prompts | 1h20 | | 18:00 | Fin de la journée |

1. Glossaire et analogie

Mot	Définition simple
Prompt	Le message que vous envoyez à l'IA.

Mot	Définition simple
Prompt système	Un prompt « fixe » qui définit la personnalité/règles de l'IA pour toute une conversation.
Prompt utilisateur	Votre demande spécifique à un moment donné.
Few-shot	Donner 2-3 exemples dans le prompt avant la vraie question. Comme une formation accélérée.
Zero-shot	Poser la question directement, sans exemple.
Skill	Un prompt prédéfini réutilisable, déclenché par un mot-clé.
Subagent	Un agent isolé avec son propre contexte, lancé pour une tâche précise.
Idempotent	Qui donne le même résultat à entrée identique.

Analogie : le freelance

Imaginez que vous engagez **un freelance senior** pour la première fois.

- Brief **vague** « *Tu peux me faire un site web ?* » → résultat **probablement médiocre**.
- Brief **précis** « *Site vitrine, 4 pages, cible PME, budget 3 k€, design moderne sobre, livrable Webflow, deadline 3 semaines, j'aime ce style : [URL]* » → résultat **utilisable**.

Un LLM = un freelance senior qui travaille à la milliseconde. Mais le **brief** reste 80 % du résultat.

2. Le canevas ROCDC-IO

C'est un acronyme à retenir. Chaque lettre = une section de votre prompt.

Lettre	Signification	Question à se poser
R	Rôle	Qui doit jouer le rôle ? Quel expert ?
O	Objectif	Quel est le résultat attendu, en 1 phrase ?
C	Contraintes	Quelles règles techniques ? Que NE PAS faire ?
D	DoD (Definition of Done)	Comment savoir que c'est fini ?
C	Contexte	Quelles infos l'IA doit connaître ?
I/O	Input/Output	Quel format d'entrée et de sortie ?

Exemple complet, commenté ligne par ligne

```
In [ ]: # =====
# Exemple : un prompt structure pour generer une route FastAPI
# =====
```

```

PROMPT = """
[ROLE]
Tu es un ingénieur Python senior, spécialiste de FastAPI et Pydantic.
Tu écris du code propre, type, et toujours teste.

[OBJECTIF]
Écris une route GET /users/{id} qui retourne un utilisateur depuis la base.

[CONTRAINTES]
- Python 3.11 minimum
- Type hints stricts partout (compatible mypy --strict)
- Asynchrone (async def)
- Pas de nouvelle dépendance hors : fastapi, pydantic, sqlalchemy
- Si l'utilisateur n'existe pas, retourner HTTP 404 avec un message clair
- Valider la réponse avec un modèle Pydantic UserResponse

[DEFINITION OF DONE]
- La route compile et se monte sur l'app sans erreur
- 3 tests pytest : cas nominal (id existe), cas 404, cas id invalide (lettres)
- Couverture >= 90 % sur le fichier produit

[CONTEXTE]
Les modèles SQLAlchemy sont déjà définis dans models.User
(champs : id: int, email: str, name: str, is_active: bool).
La session DB est injectée via Depends(get_session) défini dans deps.py.

[FORMAT DE SORTIE]
Réponse en 2 blocs Markdown :
1. routes/users.py - code complet et prêt à coller
2. tests/test_users.py - tests pytest correspondants
"""

# On peut afficher pour vérifier
print(PROMPT)
print(f"\nLongueur du prompt : {len(PROMPT)} caractères ≈ {len(PROMPT)//4} tokens")

```

Décortiquons chaque section

[ROLE] — Pourquoi ? On dit à l'IA « *sois cet expert* ». Résultat : elle utilise le **vocabulaire** et les **réflexes** de cet expert.

- ✗ Sans rôle : « *Voici une fonction pour récupérer un user...* »
- ✓ Avec rôle « ingénieur Python senior » : « *Voici la route FastAPI, avec validation Pydantic et gestion 404 idiomatique...* »

[OBJECTIF] — Pourquoi ? Une seule phrase, **active**. Force l'IA à rester focus.

[CONTRAINTES] — Pourquoi ? C'est ce qui empêche l'IA d'inventer (par ex. d'importer une librairie qui n'existe pas). Plus vous êtes précis, **moins** elle hallucine.

[DoD] — Pourquoi ? Si vous ne dites pas comment évaluer le résultat, l'IA fait ce qu'**elle** **pense** être bien. Si vous dites « *couverture ≥ 90 %* », elle vise cet objectif.

[CONTEXTE] — Pourquoi ? Si vous oubliez que `get_session` existe déjà, l'IA va le réinventer (et différemment). En lui disant « *ça existe déjà ici* », vous évitez la duplication.

[FORMAT DE SORTIE] — Pourquoi ? Si vous voulez juste copier-coller, dites « *code en bloc Markdown, sans explication* ». Si vous voulez comprendre, dites « *explique d'abord, puis code* ».

Exercice rapide : à vous

Voici 2 prompts. **Lequel vaut mieux et pourquoi ?**

Prompt A :

« *Ecris une fonction qui calcule la moyenne d'une liste.* »

Prompt B :

« *Tu es un dev Python senior. Ecris la fonction `mean(values: List[float]) -> float` qui retourne la moyenne arithmétique d'une liste. Lève `ValueError` si la liste est vide. Type hints obligatoires. Ajoute 3 tests pytest : cas nominal, liste vide, liste à 1 élément.* »

► Cliquez pour voir la réponse

3. Itérable vs Idempotent

Deux familles de prompts qu'il faut savoir reconnaître.

Prompt itérable

Définition : un prompt qu'on applique à **N entrées différentes** dans une boucle.

Exemple : « *Pour chaque fichier suivant, génère une docstring Google-style. Format de sortie : `{"fichier": "...", "docstring": "..."}` »*

Caractéristiques :

- Format de sortie **strict** (JSON, CSV).
- Pas de question à l'utilisateur en cours de route.
- Bonne candidate pour un **script** qui parse les sorties.

Prompt idempotent

Définition : un prompt qui, appelé **2 fois** avec la **même entrée**, donne le **même résultat**.

Pourquoi c'est crucial : pour les pipelines CI/CD, vous voulez que `pytest --gen-tests` produise les **mêmes** tests à chaque exécution.

Comment forcer l'idempotence ?

- Mettre `temperature=0` (= choisir le mot le plus probable, pas un mot au hasard).
- Demander un **format JSON strict**.
- Si possible, fournir un **seed** au modèle.
- Vérifier en lançant 5 fois et en comparant.

Anti-pattern : le prompt « créatif »

Terminer un prompt par « *sois créatif* » ou « *amuse-toi* » **casse** la reproductibilité. À bannir dès qu'on est en contexte dev.

```
In [ ]: # =====
# Démo : vérifier qu'un prompt est idempotent
# =====
# Note : cette démo est pédagogique, on simule un client LLM.
# Dans la vraie vie, remplacez "appeler_llm()" par votre client (OpenAI, Anthropic,

# --- Étape 1 : simuler un appel LLM (à brancher sur votre client réel)
import random

def appeler_llm_simule(prompt, temperature=0):
    """Simule un LLM. Plus la température est haute, plus il varie."""
    if temperature == 0:
        # Avec temperature=0, on retourne toujours la même chose
        return "Reponse deterministe pour : " + prompt[:20]
    else:
        # Avec temperature > 0, on ajoute du hasard
        return "Reponse aleatoire " + str(random.randint(1, 1000))

# --- Étape 2 : fonction qui teste l'idempotence
def verifier_idempotence(prompt, temperature=0, nb_essais=5):
    """Appelle N fois le même prompt et vérifie que toutes les réponses sont identi
    # On collecte les N réponses dans une liste
    reponses = []
    for i in range(nb_essais):
        r = appeler_llm_simule(prompt, temperature=temperature)
        reponses.append(r)
        print(f" Essai {i+1} : {r}")
    # set() élimine les doublons. Si tous identiques → len(set) == 1
    est_idempotent = len(set(reponses)) == 1
    return est_idempotent

# --- Test 1 : avec temperature 0 (devrait être idempotent)
print("Test avec temperature=0 :")
print(f"-> idempotent ? {verifier_idempotence('Resume ce texte', temperature=0)}")

# --- Test 2 : avec temperature 0.7 (NE devrait PAS être idempotent)
```

```
print("\nTest avec temperature=0.7 :")  
print(f"-> idempotent ? {verifier_idempotence('Resume ce texte', temperature=0.7)}")
```

4. Méta-prompting : faire écrire ses prompts par l'IA

Idée : au lieu d'écrire un prompt parfait du premier coup, **demandez à l'IA** de l'écrire avec vous.

Pattern 1 : le « prompt-builder »

Collez ceci dans ChatGPT/Claude :

Tu vas m'aider a construire un prompt parfait pour une autre IA.

Je veux qu'elle accomplisse cette tâche :
<DECRIVEZ VOTRE TACHE EN 1 PHRASE>

Avant d'ecrire le prompt final, pose-moi entre 3 et 5 questions courtes pour clarifier ce dont tu as besoin.
Une fois que j'aurai repondu, genere le prompt final structure (role, objectif, contraintes, contexte, format de sortie).

L'IA va vous poser des questions auxquelles vous **n'auriez pas pensé tout seul**. Magique.

Pattern 2 : le « critique de prompt »

Voici un prompt que je veux utiliser :

<COLLER VOTRE PROMPT ICI>

Critique-le sur 3 axes :
1. Ambigüites (qu'est-ce qui peut etre mal compris ?)
2. Risques de hallucination
3. Informations manquantes

Puis propose une version ameliee.

Pattern 3 : le « devil's advocate »

Lis ce prompt ET le code genere par l'IA.
Trouve 5 cas concrets ou ce code echouera silencieusement en production.

💡 **Astuce** : ces 3 patterns peuvent **se chaîner**. Vous construisez avec le pattern 1, vous critiquez avec le pattern 2, vous validez avec le pattern 3.

5. Skills et subagents

Skill : un prompt sauvegardé et réutilisable

Quand vous avez **bien** rédigé un prompt qui marche, **ne le réécrivez pas** à chaque fois. Enregistrez-le comme **skill**.

Un skill, c'est juste un fichier Markdown avec un **frontmatter** (entête YAML) :

```
---
name: ajouter-tests
description: Genere des tests pytest pour la fonction selectionnee.
trigger: "ajoute des tests a cette fonction"
---
```

Tu es un ingenieur tests. Pour le code donne :

1. Identifie les chemins critiques : nominal, bord, erreur.
2. Genere des tests pytest avec parametrize quand pertinent.
3. ...

Quand vous écrivez « *ajoute des tests à cette fonction* », votre IDE active automatiquement ce skill.

Subagent : un agent isolé pour une tâche longue

Quand l'utiliser :

- Lecture d'un repo de 100 fichiers (sinon, votre contexte explose).
- Recherche multi-sources (code + docs + tickets).
- Double-check indépendant (un agent revoit le travail d'un autre).

Avantage : le subagent a **son propre contexte**, qui ne pollue pas le vôtre. Vous lui posez une question, il vous donne une réponse concise, son contexte interne meurt ensuite.

Quand créer un skill ?

Règle empirique :

- ☒ Tâche **répétée** ≥ 3 fois par semaine.
- ☒ Sortie au **format strict** (commit message, ADR, story Jira...).
- ☒ Vous voulez **garantir le même comportement** à travers l'équipe.

6. Prompts spéciaux pour générer du code

7 règles d'or pour les prompts dev :

Règle 1 — Verrouiller les versions

❌ « Écris du Python. » ✅ « Python 3.11+, FastAPI 0.110, Pydantic v2. »

Règle 2 — Demander le code complet

❌ « Montre-moi la signature. » ✅ « Écris la fonction entière, imports compris. »

Règle 3 — Spécifier le framework de tests

✅ « Tests : pytest 8+, pytest-mock pour les mocks. »

Règle 4 — Anti-hallucination de librairie

✅ « Si tu n'es pas sûr qu'une librairie existe, utilise la stdlib. »

Règle 5 — Format de sortie strict

✅ « Réponds en 1 seul bloc de code Python, pas de commentaire textuel. »

Règle 6 — Préférer le diff pour les modifications

✅ « Donne-moi un diff unified au lieu du fichier complet. »

Règle 7 — Lui donner les signatures qu'il appelle

Si votre code appelle `get_session()`, collez sa **signature** dans le prompt. Sinon, l'IA invente.

Anti-hallucination : la check-list

- ✅ Joindre la signature exacte des fonctions appelées.
- ✅ Citer les imports disponibles (`requirements.txt`).
- ✅ Pour les API publiques, coller un extrait de la doc.
- ✅ Demander à l'IA de **lister les hypothèses** qu'elle a faites.

```
In [ ]: # =====
# Exemple : prompt anti-hallucination
# =====

PROMPT_ANTI_HALLU = """
[CONTEXTE - signatures existantes a respecter]
def get_session() -> AsyncSession:
    \"\"\"Retourne une session DB asynchrone.\"\"\"

class User(SQLModel):
    id: int
    email: str
```



```

name: str
is_active: bool

[OBJECTIF]
Ecris une fonction async `get_user_by_email(email: str)` qui retourne
l'utilisateur correspondant ou None si introuvable.

[CONTRAINTES]
- Python 3.11+
- Utilise get_session() ci-dessus (n'invente pas une autre fonction)
- Pas de nouvelle dependance hors sqlalchemy
- Tres important : avant ta reponse, liste explicitement
  les HYPOTHESES que tu as faites (par ex. sur la connexion DB).

[FORMAT]
1. Section "HYPOTHESES" (3-5 puces)
2. Section "CODE" (bloc Python)
"""

print(PROMPT_ANTI_HALLU)
# Astuce : forcer L'IA a lister ses hypotheses la rend beaucoup plus prudente.

```

TP 2 — Bibliothèque de prompts dev

Durée : 1h20 — Modalité : individuel puis revue à 2

Objectif

Construire **4 prompts** réutilisables au format Markdown + frontmatter, pour les 4 tâches dev les plus courantes :

1. **Refactoring** — refactorer une fonction selon SOLID.
2. **Tests** — générer des tests pytest avec couverture cible.
3. **Documentation** — docstrings Google-style + README minimal.
4. **Review** — revue de code (sécurité, perf, lisibilité, dette).

Cahier des charges

Chaque prompt doit :

- Respecter le canevas **ROCDC-IO**.
- Être **idempotent** (temperature=0 → même sortie).
- Inclure **3 cas de test** mentaux : « si je donne une fonction de X lignes, je m'attends à Y ».

Phase 1 (60 min) — Vous écrivez

Créez 4 fichiers dans un dossier `mes_prompts/` :

- `mes_prompts/refactor.md`
- `mes_prompts/tests.md`
- `mes_prompts/doc.md`
- `mes_prompts/review.md`

Phase 2 (20 min) — Revue en binôme

Avec un voisin, échangez vos prompts. Pour chacun :

- Identifiez **1 chose claire** (à garder).
- Identifiez **1 ambiguïté** (à corriger).
- Notez votre version définitive.

Livrables

- Le dossier `mes_prompts/` complet.
- Un `README.md` qui explique comment activer ces prompts dans Continue/Cursor.

📁 Corrigés et exemples de référence dans
`corriges/jour1/CORRIGE_chapitre2.ipynb` .

Synthèse de l'après-midi

- **ROCDC-IO** : Rôle, Objectif, Contraintes, DoD, Contexte, I/O.
- **Itérable** = boucle ; **Idempotent** = temperature=0, sortie déterministe.
- **Méta-prompting** : faire écrire ses prompts par l'IA elle-même.
- **Skills** = prompts sauvegardés ; **subagents** = sous-tâches isolées.
- **7 règles spécifiques au code** : versions, format strict, anti-hallucination.

🎓 **Fin du Jour 1** ! Demain : on s'attaque au **RAG** et au protocole **MCP** —
comment donner à l'IA accès à VOS documents et à VOS bases de données.